

Project 2 Proposal

MemoryWeaver

AI-Powered Photo & Voice Memory Journal

Course: Web Systems (CS 651)

Project: Social Network X Photos & Audio + Google Gemini + Google PaaS + Database

Group 4: Sarmad Habib(vf3352), Anas Niaz(te7008), Muhammad Sufiyan(ev8962)

1. App Purpose / Description

Application Name: **MemoryWeaver**

MemoryWeaver is a personal digital journal that combines photos from a social platform with voice-recorded memories to create rich, AI-enhanced narratives. The app helps users preserve and reflect on moments by:

1. Pulling photos from Google Photos (Social Network X) — users connect their account and select albums or recent photos.
2. Recording voice memos in-app — users describe the context, feelings, or story behind each photo.
3. Processing with Google Gemini — each photo is analyzed for content, mood, and visual elements (output XXX); each audio is transcribed and summarized (output YYY); optionally, both are combined into a cohesive narrative (output ZZZ).
4. Storing in Google Firestore — all results are persisted and keyed by photo/audio URLs for retrieval and display.
5. Presenting results — users see a timeline/journal view with their photos, Gemini-generated descriptions, voice transcriptions, and combined narratives.

How each component serves the purpose:

Component	Role
Photos (Google Photos)	Source of visual memories; each image is analyzed by Gemini for description, mood, and context.
Audio (in-app recording)	User-provided context; Gemini transcribes and summarizes to complement the visual analysis.
Google Gemini	Non-trivial processing: image analysis (XXX), audio transcription/summary (YYY), and optional narrative fusion (ZZZ).
Google Firestore	Stores { user, photoUrl, output XXX }, { user, audioUrl, output YYY }, and optional { user, output ZZZ }; enables deduplication and retrieval for the journal view.
Google PaaS (Cloud Run)	Hosts the backend; keeps API keys secure; serves the SPA.

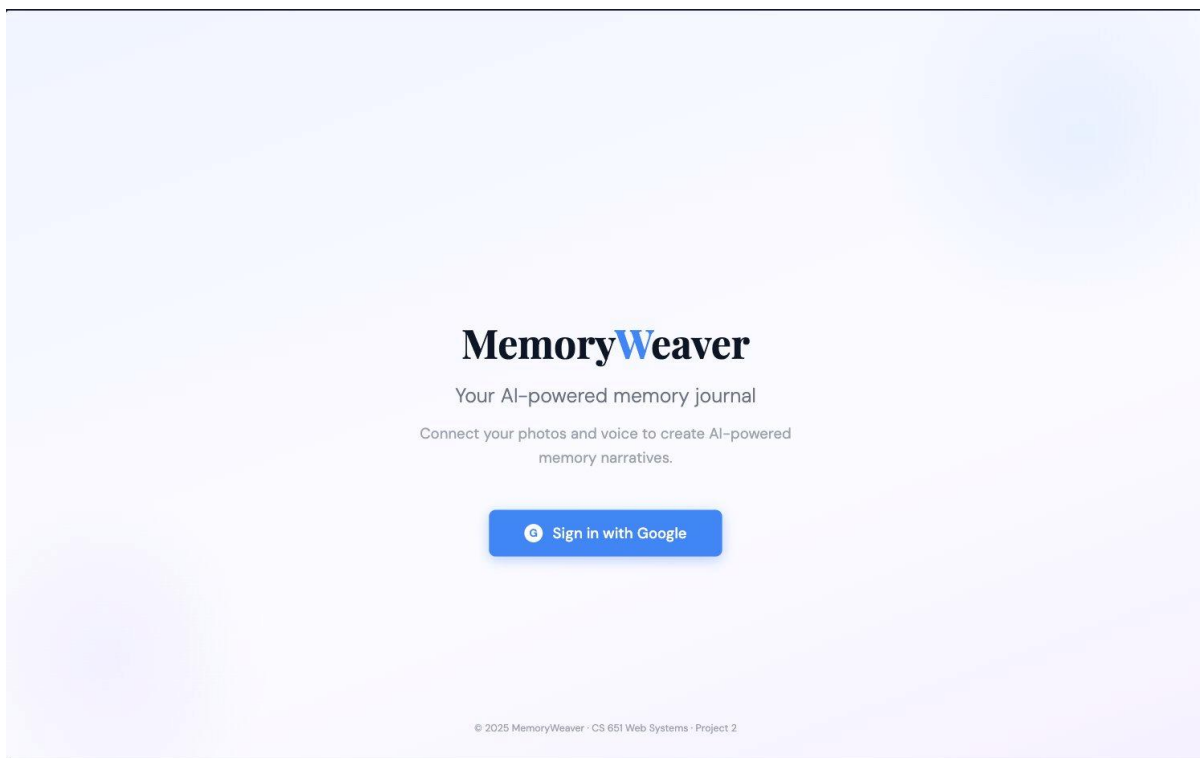
User experience: Any authenticated user (not just the developer) can log in with Google, grant photo access, record voice memos, and view their AI-enhanced journal. The app is launched from the Project 1 static site via a link.

2. Mockup Interfaces

The following screens were designed to match the proposal specifications. Each screen is 1440 × 900 px (desktop).

Screen 1: Login / Landing

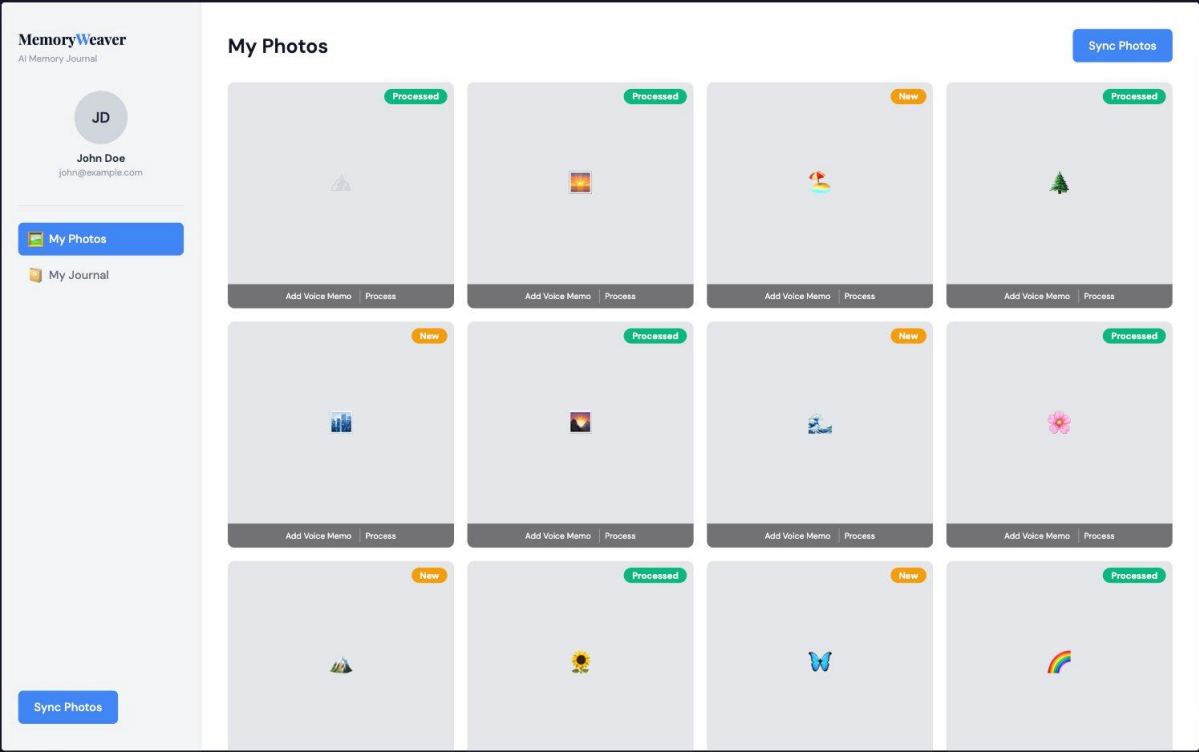
- Header: "MemoryWeaver" logo and tagline.
- CTA: "Sign in with Google" button to initiate OAuth.
- Brief description: "Connect your photos and voice to create AI-powered memory narratives."



Screen 1: Login / Landing

Screen 2: Photo Gallery (Post-Login)

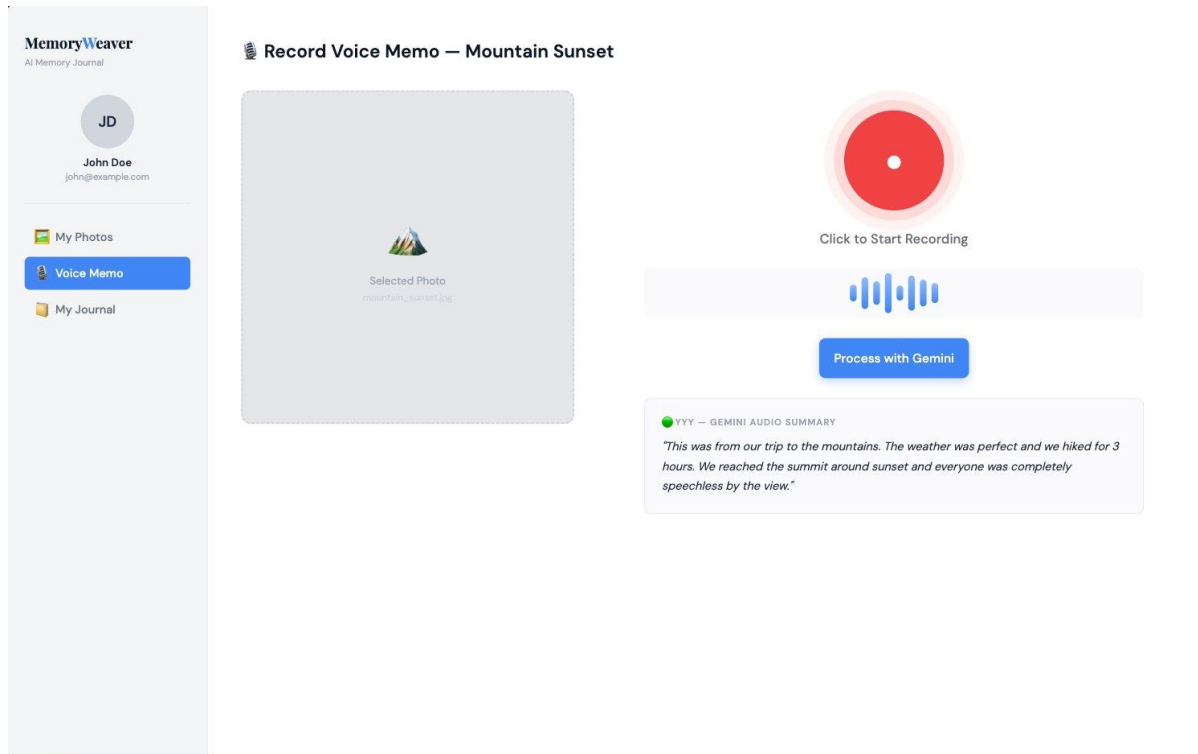
- Sidebar: User avatar, "My Photos" and "My Journal" navigation.
- Main area: Grid of photos fetched from Google Photos (thumbnails with URLs).
- Actions: "Sync Photos" button; each photo has "Add Voice Memo" and "Process" actions.
- Status: Indicator for photos already in database (Processed) vs. new (New).



Screen 2: Photo Gallery (Post-Login)

Screen 3: Voice Recording

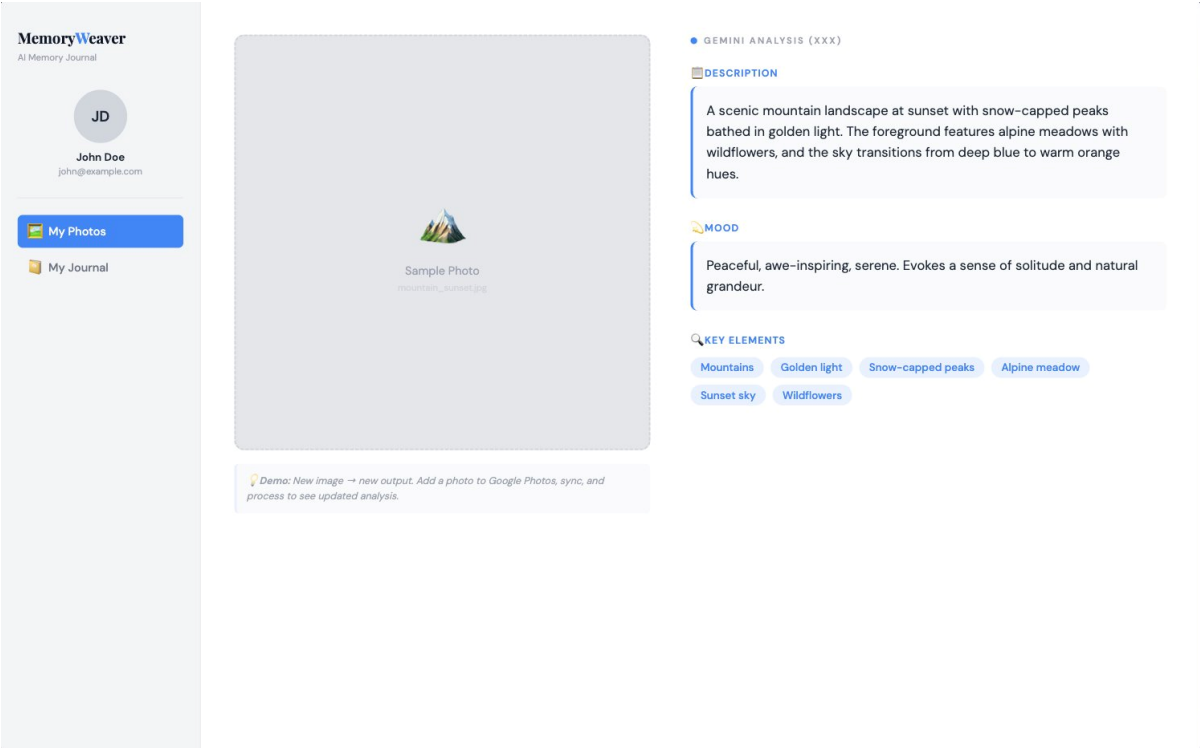
- Layout: Selected photo displayed; large "Record" button; waveform during recording.
- Actions: Start/Stop recording; "Process with Gemini" button.
- Display: After processing, show YYY (transcription + summary) below the recording controls.



Screen 3: Voice Recording

Screen 4: Photo Processing Results (XXX)

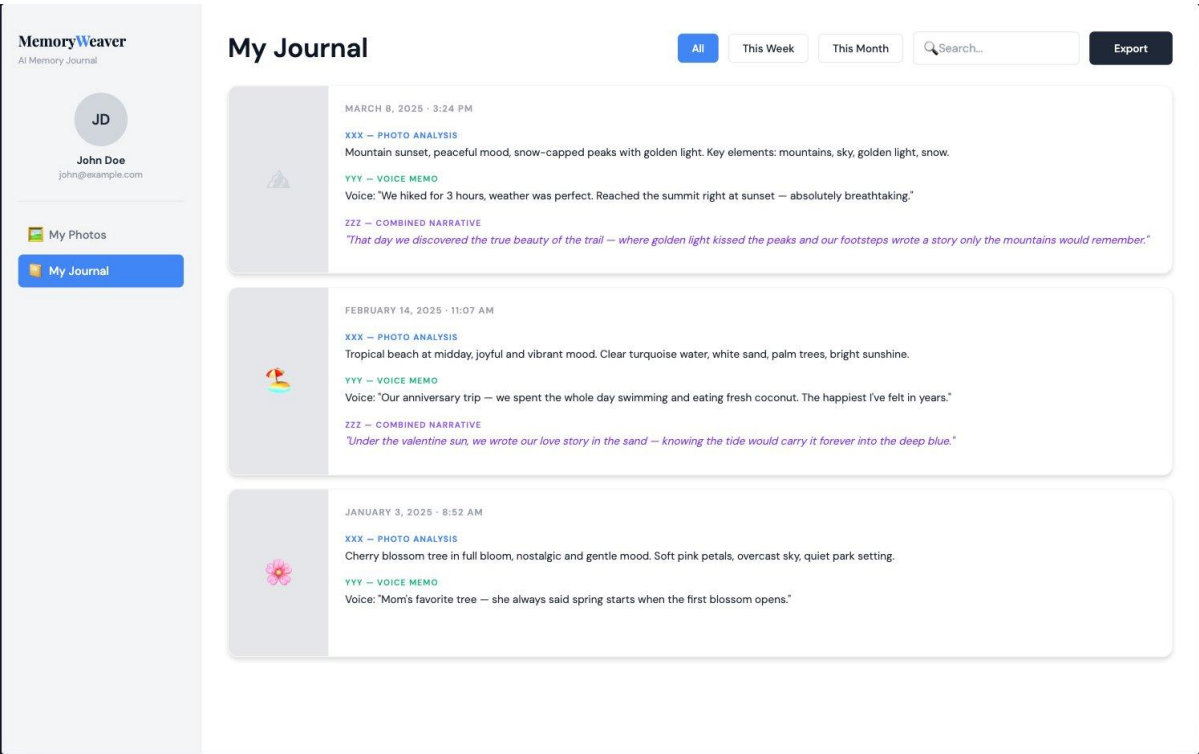
- Layout: Photo on left; Gemini output XXX (description, mood, key elements) on right.
- Data source: Retrieved from Firestore by photo URL.
- Demo link: "New image → new output" — add a new photo to Google Photos, sync, process, and see updated XXX.



Screen 4: Photo Processing Results (XXX — Gemini Analysis)

Screen 5: Journal / Timeline View

- Layout: Chronological list of entries.
- Each entry shows: photo thumbnail, XXX (Gemini photo analysis), YYY (Gemini audio summary) if voice memo exists, ZZZ (optional combined narrative) if both were processed.
- Actions: Filter by date, search, export.



Screen 5: Journal / Timeline View

Screen 6: Database View (Admin/Demo)

- Purpose: For demonstration — app-generated view of Firestore data structure.
- Shows: Document structure for photoResults, audioResults, and combinedResults collections; sample queries.

MemoryWeaver

AI Memory Journal

JD

John Doe

john@example.com

My Photos

My Journal

Database

Firestore Data Structure

Demo view — document collections and sample structure

photoResults

photoResults

```
{
  "user": "john@email.com",
  "photoUrl": "https://...",
  "output": "XXX...",
  "timestamp": "2025-03-08T..."
}
```

audioResults

audioResults

```
{
  "user": "john@email.com",
  "audioUrl": "https://...",
  "output": "YYY...",
  "timestamp": "2025-03-08T..."
}
```

combinedResults

combinedResults

```
{
  "user": "john@email.com",
  "photoUrl": "https://...",
  "output": "ZZZ...",
  "timestamp": "2025-03-08T..."
}
```

SUPPORTED FIRESTORE QUERIES

.where("user", "==", uid)

.where("photoUrl", "in", photoUrls)

.where("audioUrl", "==", url)

.orderBy("timestamp", "desc")

.limit(20)

Logging:

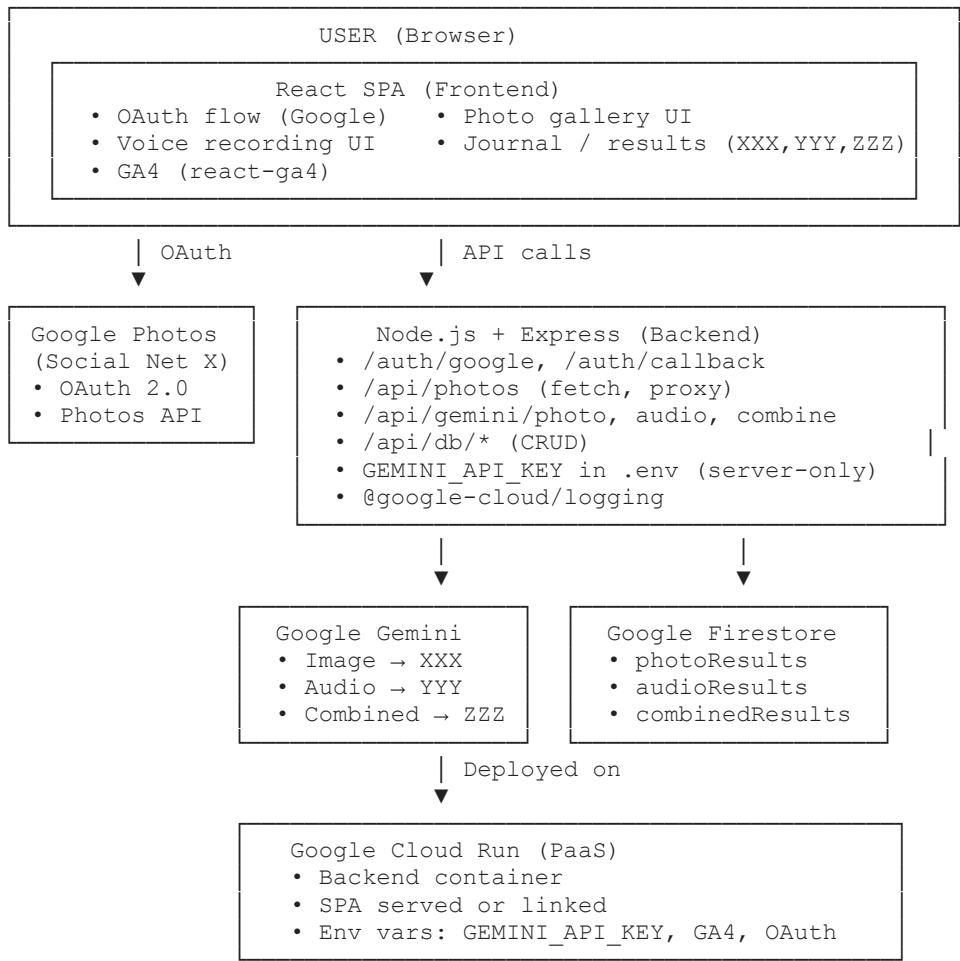
Every API call (Google Photos, Gemini, Firestore) is logged via @google-cloud/logging with endpoint, timestamp, and success/error status. Stored in Cloud Logging for debugging and auditing.

Screen 6: Database View — Firestore Data Structure (Admin/Demo)

3. System Diagram

The following diagram shows the high-level architecture of MemoryWeaver, including the React SPA frontend, Node.js + Express backend, and all Google Cloud services.

Architecture Overview:



Flow of Control Diagram

The diagram below shows the complete data flow between all components, including photo fetching, audio processing, Gemini API calls, and Firestore storage.

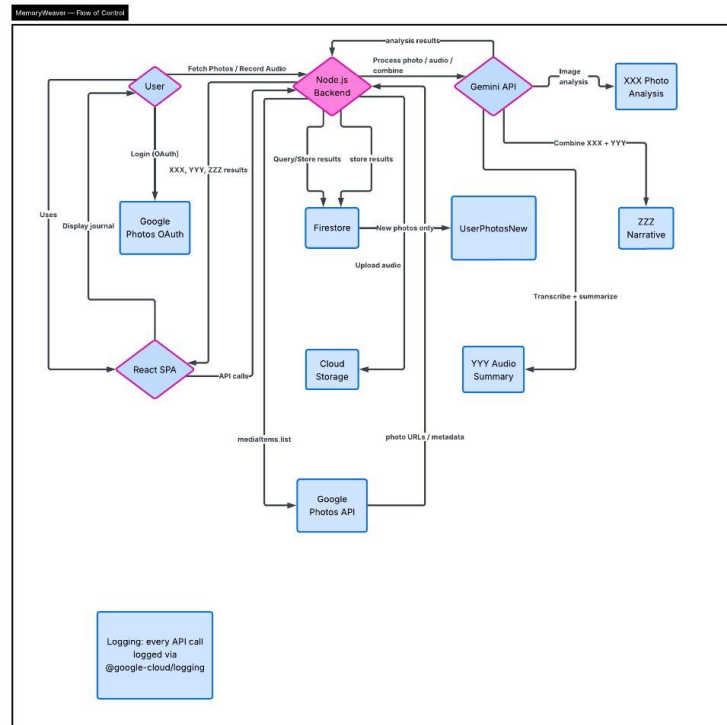


Figure: MemoryWeaver — Flow of Control Diagram (LucidChart)

4. Flow of Control Details

Steps 1–2: Retrieve User Photos

Step	Action	Technical Details
1	User clicks "Sign in with Google"	React initiates OAuth flow; redirect to Google OAuth consent.
2	User grants access	Callback to /auth/callback; Exchange code for tokens; Store tokens in session/secure cookie.
3	Fetch photos	GET /api/photos → Express calls Google Photos API (mediaitems.list); Returns list of { id, baseUrl, mimeType }.
4	Log	@google-cloud/logging: Log { action: "GooglePhotos_Fetch", endpoint: "mediaitems.list", timestamp, success }.

Data structures: `Photo[] = [{ id, baseUrl, mimeType }]`

Step 3: Photo Processing

Step	Action	Technical Details
3a	Filter new photos	Query Firestore: <code>photoResults.where("photoUrl", "in", photoUrls)</code> ; Exclude URLs already in DB → <code>UserPhotosNew[]</code> .
3b	For each image in <code>UserPhotosNew[]</code>	Fetch image bytes (or base64 URL); Call Gemini API with image + prompt ("Describe this image, its mood, and key elements").
3c	Store result	<code>Hash[photoUrl] = { userName, photoUrl, output XXX, timestamp }</code> → Firestore <code>photoResults</code> collection.
3d	Log	Log each <code>Gemini_Photo_Process</code> and <code>Firestore_Add</code> with endpoint, timestamp, success.

API: `POST /api/gemini/photo` — `body: { photoUrl, imageData }`; **returns** `{ output: XXX }`

Data structures: `XXX = string (description/mood/elements)`; Firestore doc: `{ user, photoUrl, output XXX }`

Step 4: Audio Processing

Step	Action	Technical Details
4a	Gather audio	User records in-app via MediaRecorder API; Blob sent to backend.
4b	Store media (if app-created)	Upload to Google Cloud Storage → get audio URL; or use base64 inline for small payloads.
4c	Process with Gemini	<code>POST /api/gemini/audio</code> — send audio URL or base64; Gemini transcribes and summarizes → output <code>YYY</code> .

4d	Store	Hash[audioUrl] = { userName, audioUrl, output YYY } → Firestore audioResults collection.
4e	Log	Log Gemini_Audio_Process and Firestore_Add.

Data structures: `YYY = string (transcription + summary)`; Firestore doc: { user, audioUrl, output YYY }

Step 5: Optional Combined Processing

Step	Action	Technical Details
5a	Combine XXX and YYY	POST /api/gemini/combine — send { photoOutput: XXX, audioOutput: YYY }; Gemini generates narrative ZZZ.
5b	Store	{ user, output ZZZ } → Firestore combinedResults collection.
5c	Log	Log Gemini_Combine and Firestore_Add.

Data structures: `ZZZ = string (combined narrative)`.

Step 6: Present Results

Step	Action	Technical Details
6a	Fetch stored records	GET /api/journal → Query Firestore for photoResults, audioResults, combinedResults by userName.
6b	Display	React renders timeline/journal with XXX, YYY, ZZZ per entry.
6c	Log	Log Firestore_Query for journal fetch.

Flow of Control — Logging

Every API call (Google Photos, Gemini, Firestore) is logged via `@google-cloud/logging` with the following fields:

- endpoint — the API endpoint called
- timestamp — ISO timestamp of the call
- success / error — result status
- action — descriptive label (e.g., "GooglePhotos_Fetch", "Gemini_Photo_Process", "Firestore_Add")

Complete Flow Summary

6. User → Login (OAuth) → Google Photos OAuth

7. User → Fetch Photos → Backend → Google Photos API (mediaItems.list) → photo URLs/metadata
1. Backend → Query Firestore → filter new photos → UserPhotosNew[]
2. Backend → For each new photo → Gemini API → XXX (image analysis) → Backend → Store XXX → Firestore
3. User → Record Audio → Backend → Cloud Storage (upload audio) → audio URL
4. Backend → Process audio → Gemini API → YYY (transcribe + summarize) → Backend → Store YYY → Firestore
5. Backend (optional) → Combine XXX + YYY → Gemini API → ZZZ (narrative) → Backend → Store ZZZ → Firestore
6. User → View Journal → React SPA → GET /api/journal → Backend → Query Firestore → results → React SPA → Display